ST PHILOMENA COLLEGE (AUTONOMOUS), PUTTUR
DEPARTMENT OF COMPUTER SCIENCE

DETAILED DESIGN DOCUMENT

"AI Crop Disease Detection and Treatment System"

Submitted By:

Shruthy R.S	A.V. Ancilla Mary	Jeevitha
(U05PH23C0198)	(U05PH23C0201)	(U05PH23C0184)

Submitted To:

Internal Guide

Dr.Vinayachandra
Dept. of Computer Science

St. Philomena college(Autonomous)

Head of Dept

Dr.Vinayachandra
Dept. of Computer Science

St. Philomena college(Autonomous

1. Introduction

This Detailed Design Document (DDD) provides a comprehensive low-level design of the AI Crop Disease Detection and Treatment System. It refines the system design described in the SDD by specifying the internal design of each functional module, including module decomposition, structure charts, data structures, and flowcharts.

The system allows any user to directly upload a crop leaf image through a web browser — no login is required. The uploaded image is processed by a Convolutional Neural Network (CNN) to detect the disease, and the system returns the detected disease name along with treatment recommendations. A separate Admin interface (login required) manages disease and treatment data.

2. Applicable Documents

The documents used during the preparation of this Detailed Design Document are:

- System Requirements Specification (SRS) — AI Crop Disease Detection and Treatment System
- System Design Description (SDD) — AI Crop Disease Detection and Treatment System
- Database Design Document (DDD) — AI Crop Disease Detection and Treatment System

3. Structure of the Software Package

The functional components identified during system design are:

- Functional Component 1: Farmer (User) Module — Direct image upload and result display (no login required)
- Functional Component 2: Admin Module — Manages disease, treatment, and crop data (login required)
- Functional Component 3: Image Processing Module — Preprocesses uploaded images for AI analysis
- Functional Component 4: AI Disease Detection Module — CNN-based crop disease classification
- Functional Component 5: Recommendation Module — Retrieves and displays treatment suggestions
- Functional Component 6: Database Module — Manages all persistent data storage

4. Modular Decomposition of Components

4.1 Modules of Component 1: Farmer (User) Module

4.1.1 Design Assumptions

No login or registration is required. Any user can access the system via a web browser, upload a crop leaf image, and receive disease detection results immediately. The system is publicly accessible.

4.1.2 Identification of Modules

- Module 1.1: Image Upload
- Module 1.2: View Disease Detection Result

4.1.3 Structure Chart

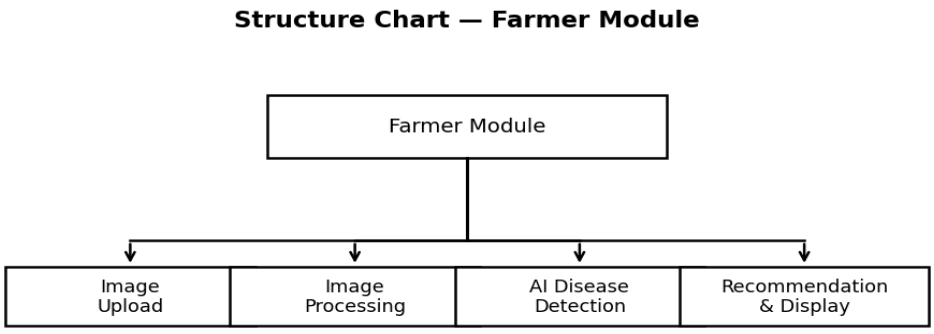

Fig 4.1.3: Structure Chart — Farmer Module

4.1.4 Data Structures Shared Among Modules

Data Element	Type	Description	Shared With
image_path	STRING	Server path to uploaded image	Image Upload, AI Module
disease_result	STRING	Detected disease name	View Result, Recommendation

Data Element	Type	Description	Shared With
confidence_score	FLOAT (0.0–1.0)	AI model prediction confidence	View Result
treatment_info	STRING	Treatment recommendation text	View Result

4.2 Modules of Component 2: Admin Module

4.2.1 Design Assumptions

The admin has a secured login with username and password. Only the admin can add, edit, or delete disease and treatment records. Admin operations directly update the database.

4.2.2 Identification of Modules

- Module 2.1: Admin Login
- Module 2.2: Add / Edit Disease Data
- Module 2.3: Add / Edit Treatment Data
- Module 2.4: View Reports

4.2.3 Structure Chart

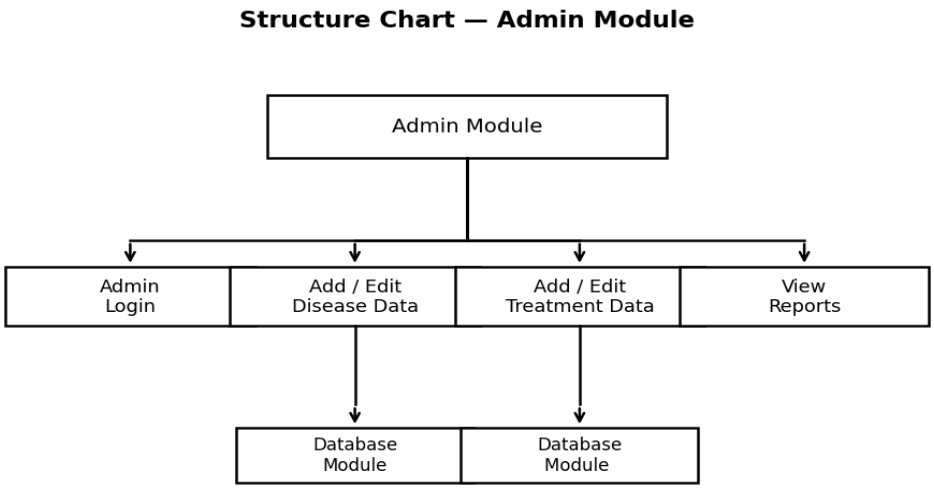

Fig 4.2.3: Structure Chart — Admin Module

4.2.4 Data Structures Shared Among Modules

Data Element	Type	Description	Shared With
admin_id	INT (PK)	Unique admin identifier	Login, Disease, Treatment
disease_id	INT (PK)	Disease record ID	Disease, Treatment, Prediction
treatment_id	INT (PK)	Treatment record ID	Treatment, Recommendation
crop_id	INT (FK)	Linked crop ID	Disease, Crop

4.3 Modules of Component 3: Image Processing Module

4.3.1 Design Assumptions

Input images must be JPG or PNG format. Images are resized to 224×224 pixels for CNN model compatibility. OpenCV and Pillow libraries are used for preprocessing.

4.3.2 Identification of Modules

- Module 3.1: Image Validation
- Module 3.2: Image Resizing and Normalization
- Module 3.3: Prepare Input Array for CNN

4.3.3 Structure Chart

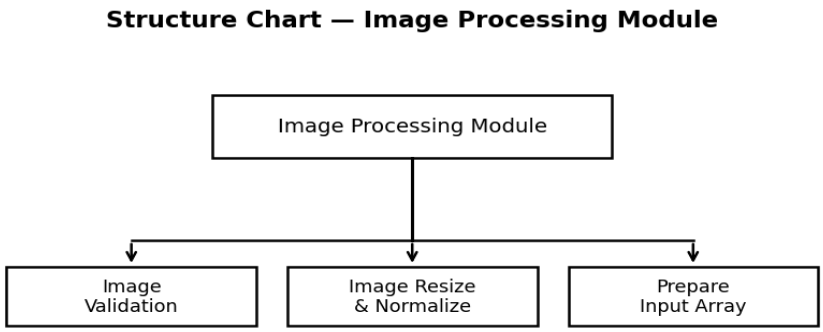

Fig 4.3.3: Structure Chart — Image Processing Module

4.3.4 Data Structures Shared Among Modules

Data Element	Type	Description	Shared With
raw_image	FILE (JPG/PNG)	Original uploaded image	Image Validation
processed_array	NumPy Array (224,224,3)	Normalized image array	AI Detection Module
image_id	INT (PK)	Stored image reference ID	Prediction Table

4.4 Modules of Component 4: AI Disease Detection Module

4.4.1 Design Assumptions

A pre-trained CNN model in TensorFlow/Keras (.h5 format) is available on the server. The model is trained on the PlantVillage dataset covering tomato, spinach, ladyfinger, brinjal, and arecanut.

4.4.2 Identification of Modules

- Module 4.1: Model Loading

- Module 4.2: Disease Prediction
- Module 4.3: Accuracy Score Computation

4.4.3 Structure Chart

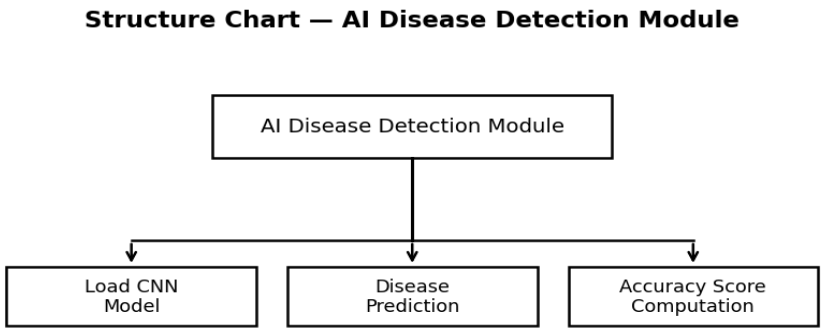

Fig 4.4.3: Structure Chart — AI Disease Detection Module

4.4.4 Data Structures Shared Among Modules

Data Element	Type	Description	Shared With
model_file	H5 / SavedModel	Trained CNN model file	Model Loading
input_tensor	NumPy Array (1,224,224,3)	Preprocessed image tensor	Image Processing Module
prediction_class	STRING	Predicted disease class label	Recommendation Module
confidence_score	FLOAT (0.0–1.0)	Prediction confidence	Result Display, Prediction Table

4.5 Modules of Component 5: Recommendation Module

4.5.1 Design Assumptions

Disease-to-treatment mapping exists in the database. If no treatment is found for a detected disease, a default advisory message is returned to the user.

4.5.2 Identification of Modules

- Module 5.1: Disease Lookup
- Module 5.2: Treatment Retrieval
- Module 5.3: Format and Display Result

4.5.3 Structure Chart

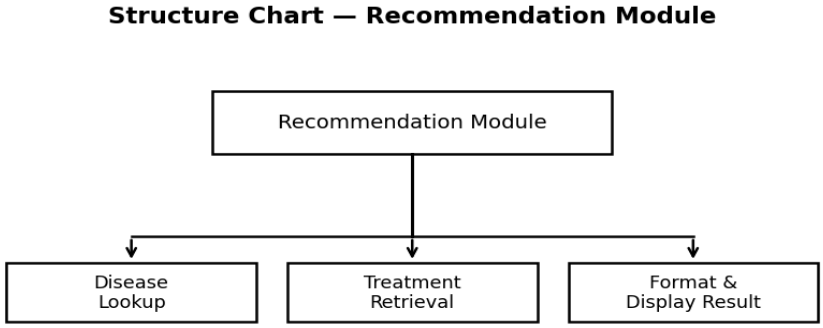

Fig 4.5.3: Structure Chart — Recommendation Module

4.5.4 Data Structures Shared Among Modules

Data Element	Type	Description	Shared With
disease_name	STRING	Detected disease name	AI Module, Database Module
treatment_info	STRING	Treatment description	Result Display
fertilizer	STRING	Recommended fertilizer	Result Display
pesticide	STRING	Recommended pesticide	Result Display

4.6 Modules of Component 6: Database Module

4.6.1 Design Assumptions

MySQL or SQLite is used. All tables are normalized to 3NF. Foreign key constraints are enforced. No User table is needed since there is no farmer login.

4.6.2 Identification of Modules

- Module 6.1: Crop and Disease Data Management
- Module 6.2: Treatment Data Management
- Module 6.3: Image and Prediction Storage

4.6.3 Structure Chart

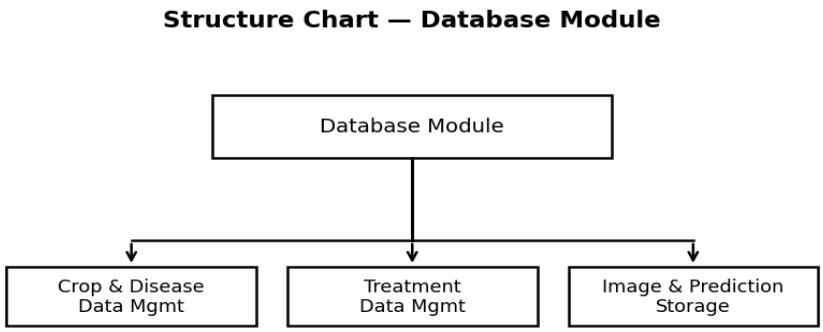

Fig 4.6.3: Structure Chart — Database Module

5. Detailed Design

5.1 Module Design of Component 1: Farmer (User) Module

5.1.1 Module 1.1 — Image Upload

5.1.1.1 Inputs:

Parameter	Source	Format	Valid Range
Crop leaf image	User file input (web browser)	JPG / PNG	Max 5MB, minimum 100×100 pixels

5.1.1.2 Procedural Details (Flow Chart):

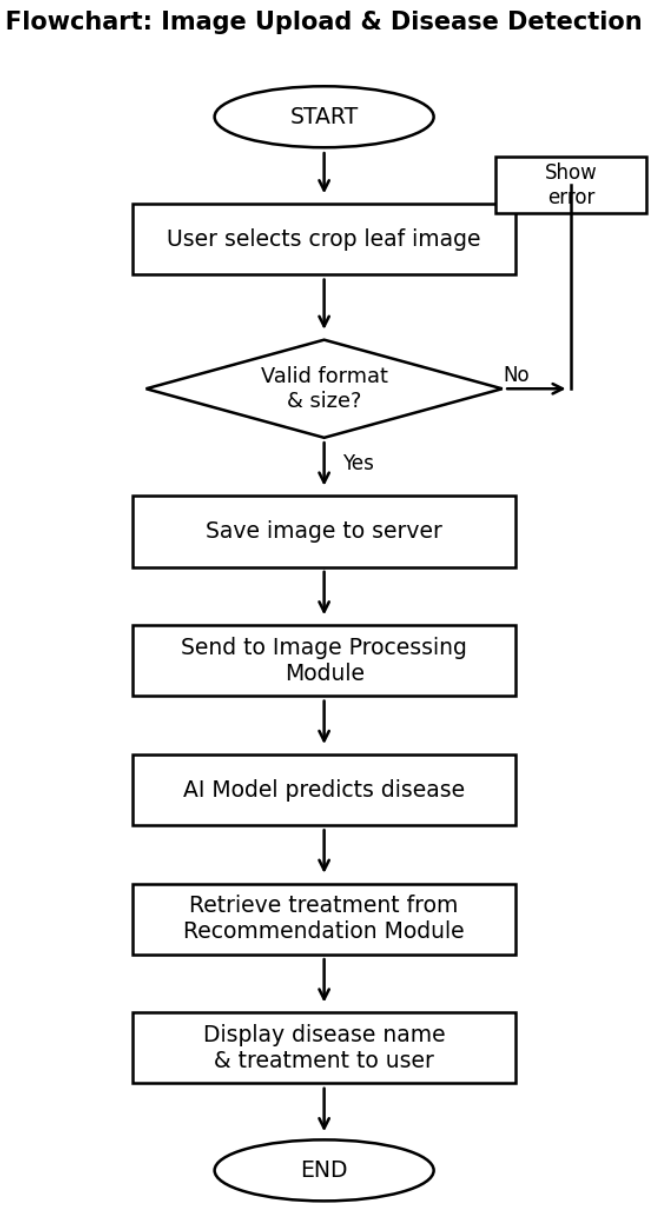

Fig 5.1.1: Flowchart — Image Upload & Disease Detection

5.1.1.3 File I/O Interfaces:

- Writes: Image file saved to /uploads/ directory on server
- Inserts: Record into Image table (Image_Path, Upload_Date)

5.1.1.4 Outputs:

- Disease name detected by AI model
- Confidence score (prediction accuracy percentage)
- Treatment recommendation (fertilizer, pesticide, prevention tips)

5.1.1.5 Implementation: Flask file upload, os.path, Pillow (PIL) for file validation

5.2 Module Design of Component 3: Image Processing Module

5.2.1 Module 3.2 — Image Resizing and Normalization

5.2.1.1 Inputs:

Parameter	Source	Format	Valid Range
Raw image	Image Upload Module	JPG / PNG file	Any resolution
Target size	System configuration	Integer tuple	(224, 224)

5.2.1.2 Procedural Details (Pseudo-code in Structured English):

Load image → Resize to (224, 224) → Convert to NumPy array (224,224,3) → Normalize pixel values by dividing by 255.0 → Expand dimensions to (1,224,224,3) → Return preprocessed array to AI module.

5.2.1.3 File I/O Interfaces:

- Reads: Image file from /uploads/ directory
- Outputs: In-memory NumPy array passed directly to AI Detection Module

5.2.1.4 Outputs: Normalized NumPy array of shape (1, 224, 224, 3)

5.2.1.5 Implementation: OpenCV (cv2.resize), PIL/Pillow, NumPy libraries

5.3 Module Design of Component 4: AI Disease Detection Module

5.3.1 Module 4.2 — Disease Prediction

5.3.1.1 Inputs:

Parameter	Source	Format	Valid Range
Preprocessed image array	Image Processing Module	NumPy Array (1,224,224,3)	Float values 0.0 to 1.0
CNN Model	Model file (model.h5)	Keras Model object	Pre-trained, loaded at startup

5.3.1.2 Procedural Details (Flow Chart):

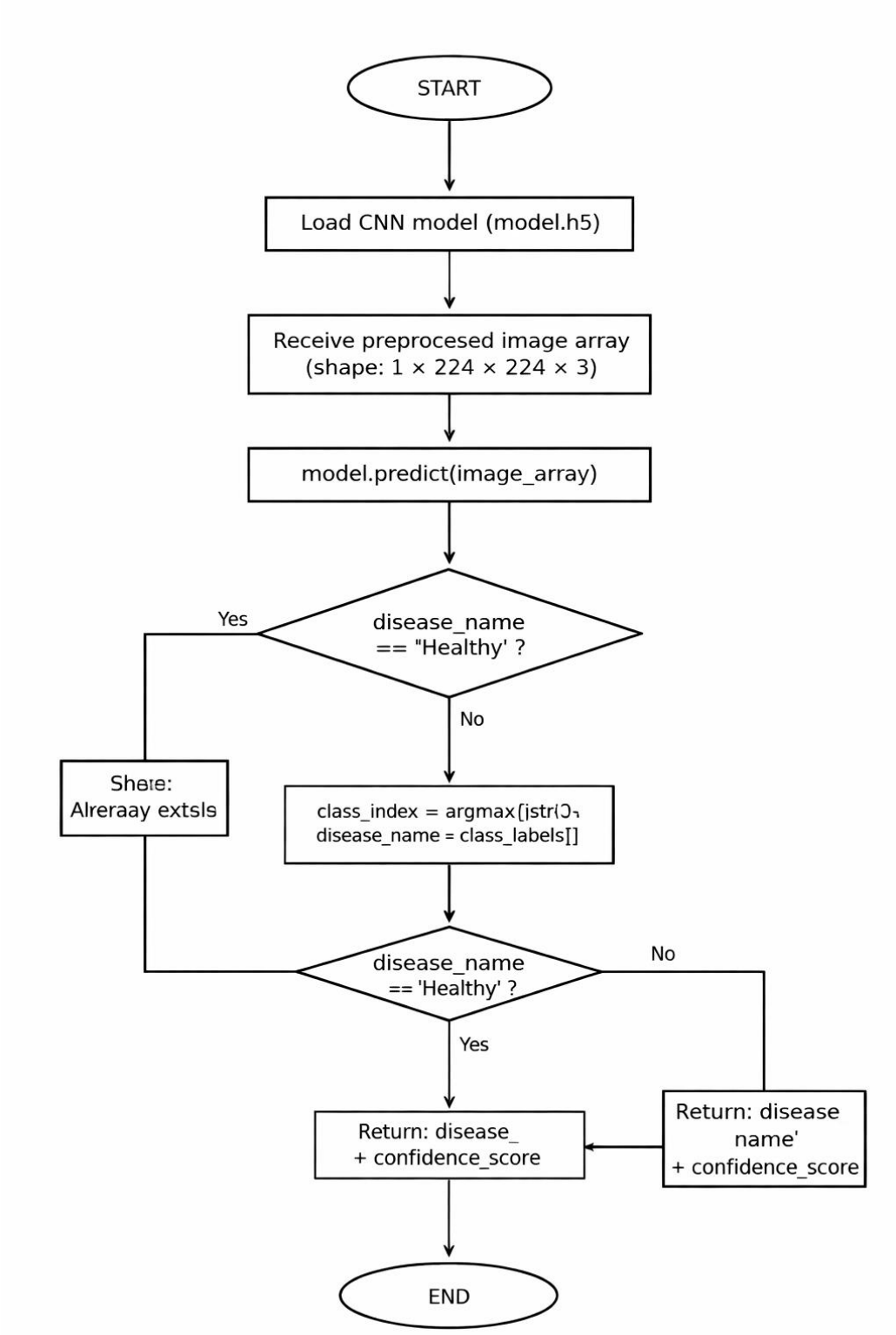

Fig 5.3.1: Flowchart — CNN Disease Prediction

5.3.1.3 Table I/O Interfaces:

- Reads: class_labels list (list of disease class names)
- Writes: Result to Prediction table (Image_ID, Disease_ID, Accuracy, Result_Label)

5.3.1.4 Outputs:

- disease_name: String (e.g., 'Tomato Late Blight')
- confidence_score: Float (e.g., 0.94 representing 94% confidence)

5.3.1.5 Implementation: TensorFlow 2.x, Keras model.predict(), numpy.argmax()

5.4 Module Design of Component 5: Recommendation Module

5.4.1 Module 5.2 — Treatment Retrieval

5.4.1.1 Inputs:

Parameter	Source	Format	Valid Range
disease_name	AI Detection Module	String	Must match entry in Disease table
disease_id	Database lookup	Integer (FK)	Must exist in Disease table

5.4.1.2 Procedural Details (Flow Chart):

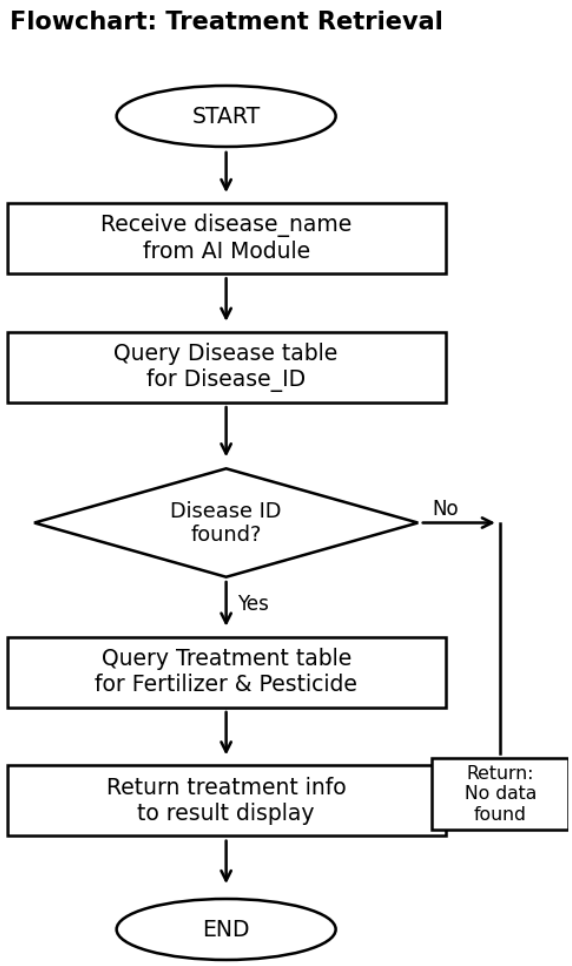

Fig 5.4.1: Flowchart — Treatment Retrieval

5.4.1.3 Table I/O Interfaces:

- Reads: Disease table (Disease_ID, Disease_Name)
- Reads: Treatment table (Fertilizer, Pesticide, Prevention, Dosage)

5.4.1.4 Outputs: Fertilizer, pesticide, and prevention tips displayed to user

5.4.1.5 Implementation: Flask-SQLAlchemy ORM or MySQL-connector-python

5.5 Module Design of Component 2: Admin Module

5.5.1 Module 2.2 — Add / Edit Disease Data

5.5.1.1 Inputs:

Parameter	Source	Format	Valid Range
disease_name	Admin form	String	Max 150 characters, non-empty
symptoms	Admin form	Text	Max 500 characters
crop_id	Admin dropdown	Integer (FK)	Must exist in Crop table

5.5.1.2 Procedural Details (Flow Chart):

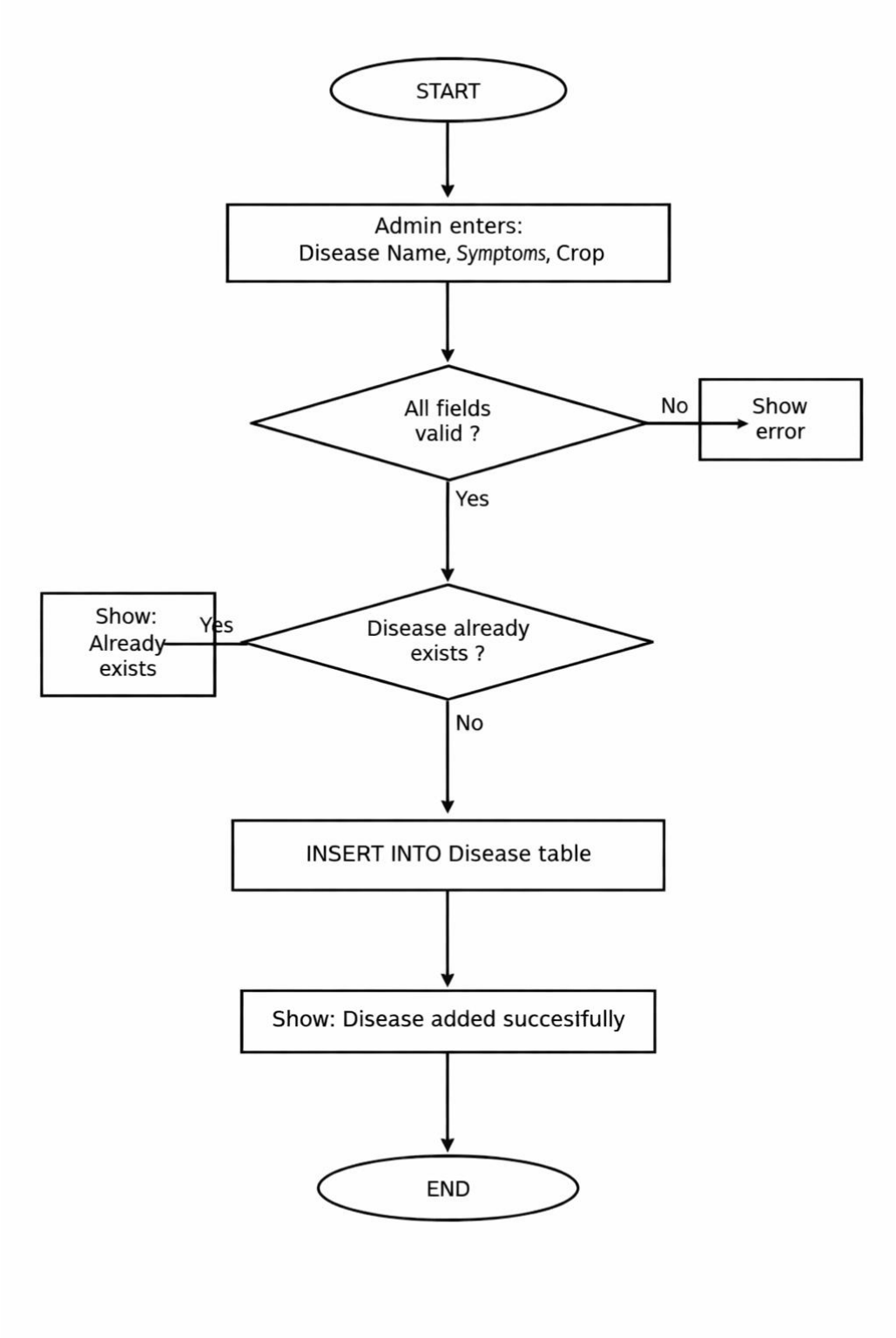

Fig 5.5.1: Flowchart — Admin Add Disease

5.5.1.3 Table I/O Interfaces:

- Reads: Crop table (for foreign key validation)
- Writes: New record into Disease table

5.5.1.4 Outputs: New Disease record inserted; success or error message displayed to Admin

5.5.1.5 Implementation: Flask route with form validation, SQLAlchemy ORM